

Practica H.264/AVC

Servicios Multimedia - UDC

Curso 2025/26

1 Codificación de video con H264

Para comprender el funcionamiento del estándar H.264, en esta práctica se realizará un estudio de diversos casos prácticos usando diferentes herramientas open source. Se presentarán los conceptos teóricos y sobre ellos se plantearán unos ejercicios y unas cuestiones a contestar. En la práctica nos centraremos en la parte de codificación del estándar, especialmente en el uso de los distintos tipos de macrobloques y perfiles de codificación.

1.1 Material a entregar

La práctica se puede realizar de forma individual o en pareja.

Como resultado de esta práctica se deberá entregar un documento de texto (PDF) a través del campus virtual con las respuestas a los ejercicios planteados, incluyendo las capturas y tablas requeridas. En el campus virtual se encuentra una plantilla en formato ODT para facilitar la redacción.

Si la práctica se hace en pareja, solo debe entregarla uno de los dos. En el fichero debe aparecer el nombre de ambos. También debe indicarse qué secuencia de video fue la utilizada durante la práctica.

1.2 Secuencias de vídeo asignadas

Para realizar la práctica se asigna a cada alumno una secuencia de video, disponible en el campus virtual, con resolución 720x398 y una duración de 20 segundos. En caso de hacerse en pareja, se escogerá una de las dos. Las secuencias fueron obtenidas a partir de la película Charade, debido a que al no reclamarse los derechos en las primeras copias esta es ahora de dominio público, por lo que puede verse y descargarse libremente (para más detalles ver <https://es.wikipedia.org/wiki/Charada>).

2 Instalación del entorno de trabajo

Para realizar esta práctica usaremos las siguientes herramientas:

- **ffmpeg**: herramienta open source de procesamiento de vídeo y audio. Nos permitirá realizar el proceso de codificación usando diferentes parámetros, así como obtener estadísticas de bajo nivel que podremos analizar.
- **vlc**: reproductor open source de vídeo y audio. Nos permitirá visualizar los vídeos codificados y los vídeos con overlay de macrobloques. Se puede usar cualquier otro similar, pero es recomendable que tenga la opción de avanzar fotograma a fotograma.

- python y algunas librerías como:
 - `numpy`
 - `opencv-python`
 - `matplotlib`
- Un script `mb_visualizer.py`, que nos permitirá generar una visualización de los distintos tipos de macrobloques usados en la codificación y sacar unas estadísticas de los mismos.

2.1 Instalación de ffmpeg

2.1.1 En linux (debian/ubuntu y similares)

Para instalar `ffmpeg` en Linux, ejecuta el siguiente comando:

```
1 sudo apt update
2 sudo apt install ffmpeg
```

Comprueba la instalación:

```
1 ffmpeg -version
```

En distribuciones con diferentes gestores de paquetes, utiliza el comando equivalente.

2.1.2 En windows

Para instalar `ffmpeg` en Windows, sigue estos pasos:

1. Descarga ffmpeg:

- Ve a la página web oficial de ffmpeg: <https://ffmpeg.org/download.html>
- En la sección “Windows”, pincha en alguna de las opciones que pone “Windows builds...”. Por ejemplo “from gyan.dev”
- Descarga la versión `ffmpeg-release-essentials.7z` en la sección de “release builds”.
- Descomprime el archivo en una carpeta conocida, por ejemplo `C:\ffmpeg`.

2. Añade ffmpeg al PATH del sistema:

- Abre el menú de Windows y busca “Variables de entorno” o “Environment Variables”
- Haz clic en “Editar las variables de entorno del sistema”
- En la ventana “Propiedades del sistema”, haz clic en “Variables de entorno”
- En “Variables del sistema”, busca la variable “Path” y selecciónala

- Haz clic en “Editar”
- Haz clic en “Nuevo” y añade la ruta completa a la carpeta `bin` de ffmpeg (ejemplo: `C:\ffmpeg\bin`)
- Haz clic en “Aceptar” en todas las ventanas

3. Reinicia la terminal:

- Cierra cualquier ventana de PowerShell o CMD que tengas abierta
- Abre una nueva ventana de PowerShell o CMD

Comprueba la instalación ejecutando:

```
1 ffmpeg -version
```

Si todo está correcto, deberías ver información sobre la versión de ffmpeg instalada. Si obtienes un error como “ffmpeg no se reconoce como un comando”, verifica que hayas añadido correctamente la ruta al PATH y que hayas reiniciado la terminal.

2.1.3 Opción alternativa: Instalación con Winget (recomendado)

Si tienes Windows 10 versión 1709 o superior, puedes usar **Winget** (el administrador de paquetes oficial de Windows) para instalar ffmpeg de forma mucho más sencilla:

1. Instala Winget (si no lo tienes):

- Si tienes Windows 11, ya viene instalado por defecto
- Si tienes Windows 10, puedes instalarlo desde la Microsoft Store buscando “App Installer”

2. Instala ffmpeg con Winget:

- Abre PowerShell o CMD como administrador
- Ejecuta el siguiente comando:

```
1 winget install ffmpeg
```

3. Verifica la instalación:

```
1 ffmpeg -version
```

Ventajas de usar Winget:

- Instalación automática sin necesidad de descargar archivos manualmente
- Actualizaciones automáticas disponibles con `winget upgrade ffmpeg`
- Desinstalación sencilla con `winget uninstall ffmpeg`
- No requiere modificar el PATH manualmente

2.2 Instalación de VLC

VLC es un reproductor multimedia versátil que utilizaremos para visualizar los vídeos codificados y analizar los fotogramas frame a frame.

2.2.1 En linux (debian/ubuntu y similares)

Para instalar VLC en distribuciones basadas en Debian/Ubuntu:

```
1 sudo apt update
2 sudo apt install vlc
```

Comprueba la instalación ejecutando:

```
1 vlc --version
```

En distribuciones con diferentes gestores de paquetes: - **Fedora/CentOS/RHEL**: `sudo dnf install vlc` o `sudo yum install vlc` - **Arch Linux**: `sudo pacman -S vlc` - **openSUSE**: `sudo zypper install vlc`

2.2.2 En windows

2.2.2.1 Opción 1: Instalación desde el sitio oficial (recomendado)

1. Descarga VLC:

- Ve al sitio web oficial: <https://www.videolan.org/vlc/>
- Haz clic en “Download VLC” en la página principal
- Descarga la versión para Windows (normalmente el instalador .exe)

2. Instala VLC:

- Ejecuta el instalador descargado (.exe)
- Sigue el asistente de instalación (las opciones por defecto suelen ser adecuadas)
- VLC se instalará automáticamente y estará disponible en el menú Inicio

3. Verifica la instalación:

- Abre VLC desde el menú Inicio o busca “VLC” en la barra de búsqueda
- También puedes verificar desde CMD o PowerShell:

```
1 vlc --version
```

2.2.2.2 Opción 2: Instalación con Winget (automática) Si tienes Windows 10 versión 1709 o superior, puedes instalar VLC automáticamente con Winget:

```
1 winget install VideoLAN.VLC
```

2.3 Instalación de `mb_visualizer.py`

Para ejecutar el script `mb_visualizer.py` necesitaremos instalar python y las librerías de python necesarias. Para ello, revisa el documento de instalación de la práctica anterior (JPEG) e instala python de alguna de las posibles opciones que se proponen.

Clona el repositorio de `mb_visualizer.py` desde GitHub:

```
1 git clone https://github.com/valentinbarral/h264_ffmpeg_mb.git
```

Una vez clonado:

```
1 cd h264_ffmpeg_mb
2 pip install opencv-python matplotlib numpy
```

La herramienta se usa así:

```
1 python mb_visualizer.py input.y4m \
2     --output-video output.mp4 \
3     --params "some ffmpeg parameters"
```

- `input.y4m`: vídeo de entrada.
- `--output-video output.mp4`: nombre base del vídeo codificado.
- `--params "..."`: cadena que se pasa tal cual como parámetros de `ffmpeg` para la codificación.

La herramienta genera:

1. `output.mp4` → vídeo codificado con los parámetros pasados.
2. `output_overlay_mb.mp4` → vídeo con overlay de tipos de bloque.
3. `output_stats.txt` → estadísticas de macrobloques.
4. `output_metrics_plot.png` → gráficos con la evolución del bitrate y el parámetro de cuantización (QP) a lo largo de la codificación.

2.4 Preparación del vídeo de referencia en formato y4m

Para poder realizar una codificación limpia, y poder comparar los resultados al variar los parámetros de codificación, necesitaremos un vídeo sin comprimir. Para ello, lo primero que haremos será convertir

el vídeo MP4 asignado a un fichero Y4M sin comprimir. Con esto tendremos un punto de origen fijo (con un tamaño establecido) que nos permitirá calcular la tasa de compresión de la codificación para cada nueva configuración. En este tipo de fichero las componentes de luminancia y crominancia se guardan en crudo, sin ninguna compresión.

$$\text{Tasa de compresión} = \frac{\text{Tamaño del fichero Y4M}}{\text{Tamaño del MP4 codificado}}$$

Para convertir el vídeo `input.mp4` a Y4M:

```
1 ffmpeg -i video_asignado.mp4 video_asignado.y4m
```

Significado de los parámetros:

- `-i video_asignado.mp4` Vídeo de entrada asignado.
- `video_asignado.y4m` Fichero de salida en formato Y4M, que incluye automáticamente la resolución y framerate del vídeo original, pero los datos sin comprimir.

A partir de este momento:

- Todas las re-codificaciones se harán desde `video_asignado.y4m`.
- En todos los comandos `ffmpeg` que codifican H264 se usará el patrón:

```
1 ffmpeg -i video_asignado.y4m \  
2 [parámetros de codificación] \  
3 salida.mp4
```

Cuando usemos el script `mb_visualizer.py`, usaremos el patrón:

```
1 python mb_visualizer.py video_asignado.y4m \  
2 --output-video salida.mp4 \  
3 --params "[parámetros de codificación]"
```

2.5 Recomendación para el trabajo con la práctica

Para que sea más sencillo probar las diferentes configuraciones, lo más sencillo es descargar y transformar el video asignado dentro de la carpeta donde se encuentra el script `mb_visualizer.py`. De esta manera no tendremos que pasar la ruta completa del video cada vez que lo queramos codificar.

3 Práctica

3.1 Macrobloques intra e inter

Un vídeo H.264 no se codifica píxel a píxel, sino en **macrobloques de 16×16**. Cada macrobloque puede subdividirse (8×8, 16×8, 8×16, 4×4...) y, para cada bloque o sub-bloque, el codificador decide:

- si lo va a predecir a partir del **mismo frame** (predicción INTRA),
- si lo va a predecir a partir de **otro frame** (predicción INTER),
- o si lo puede dar prácticamente por “copiado” (SKIP) porque el cambio es mínimo.

Así tenemos:

- **INTRA:**
 - la predicción usa píxeles vecinos del mismo frame,
 - hay modos “gruesos” (16×16) y modos “finos” (4×4, 8×8),
 - se usan mucho en zonas con detalle o cuando el movimiento es difícil de predecir.
- **INTER:**
 - la predicción usa bloques de otros frames (anteriores o posteriores),
 - se usan vectores de movimiento:
 - * 16×16 → un vector para todo el macrobloque,
 - * 16×8 / 8×16 → un vector por cada media partición,
 - * 8×8 (y sub-particiones) → vectores por bloques más pequeños cuando el movimiento es más complejo.
- **SKIP:**
 - el codificador decide que “no merece la pena” enviar información detallada,
 - el decodificador reconstruye el bloque casi únicamente a partir de la predicción,
 - se usa en zonas casi estáticas.

Con **ffmpeg** podemos activar el modo depuración para ver algunos tipos de macrobloques, aunque no todos los subtipos. Los que podemos ver son:

- **INTRA** (Rojo en el overlay) : Referencia espacial (dentro del mismo frame)
 - **I** : Intra 16x16 (Bloque grande plano)
 - **i** : Intra 4x4 / 8x8 (Bloque con detalle)
- **SKIP** (Verde en el overlay) : Copiado directo

- S : Macrobloque Skip (Copiado del frame anterior, muy eficiente)
- **INTER** (Azul en el overlay) : Referencia temporal (movimiento)
 - d : Inter 16x16 (Predicción L0)
 - D : Inter 16x16 (Predicción L1 / Bi-pred / Direct)
 - < : Partición 16x8 (Rectangular horizontal)
 - > : Partición 8x16 (Rectangular vertical)
 - X : Partición 8x8 (Sub-particionado)
 - Otras combinaciones de símbolos indican todavía más subtipos de particiones del macro-bloque.

El script `mb_visualizer.py` nos generará un vídeo con una capa de overlay en la que se pintan estos tipos de macrobloques. Si después lo abrimos con `vlc` podemos ir frame a frame (tecla `E`) y ver los distintos tipos de macrobloques en cada fotograma.

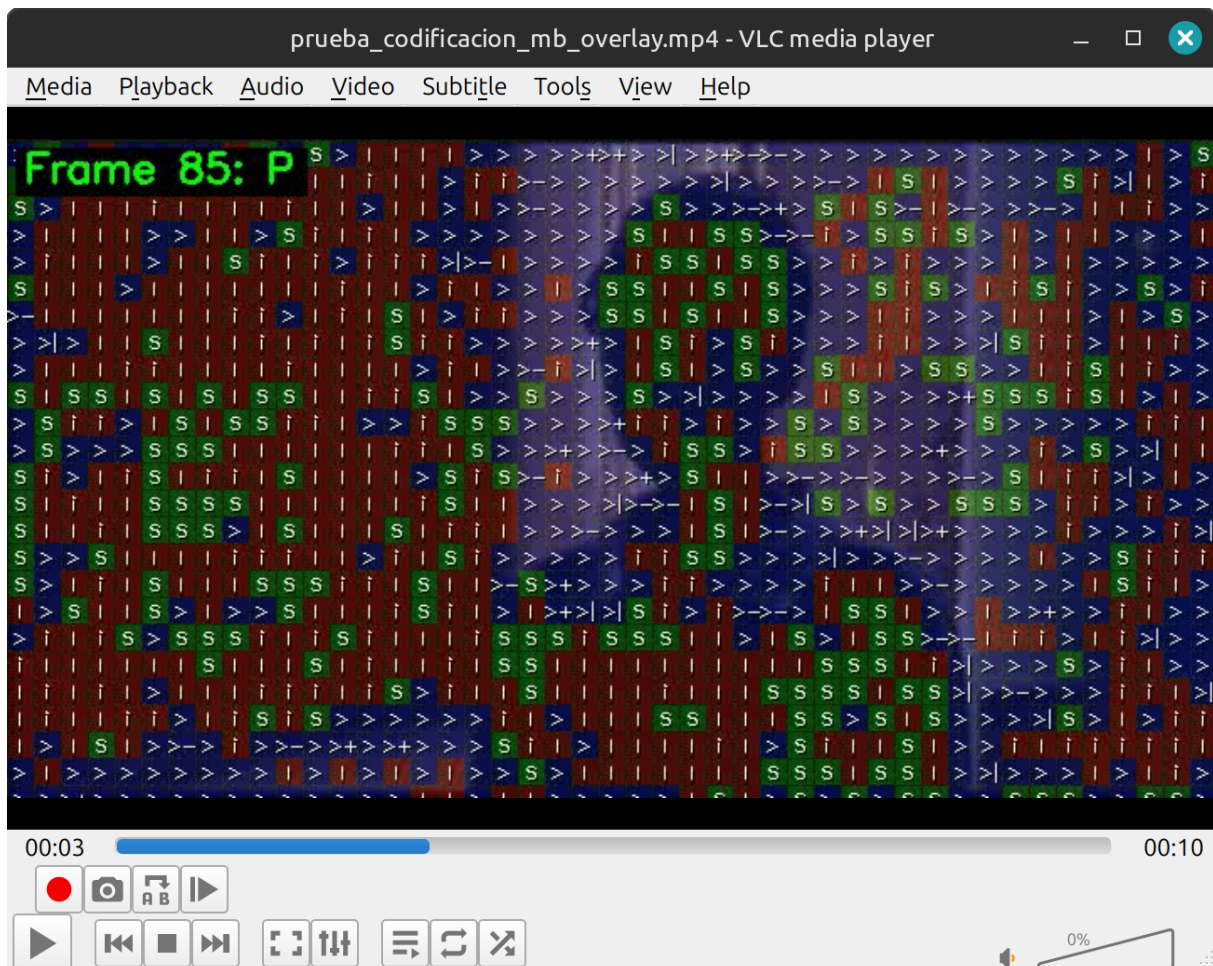


Figura 1: Overlay de macrobloques

También podremos ver en la esquina superior derecha el número y tipo de frame:

- **I-frame (Intra):**

- se codifica solo con predicción intra,
- sirve como punto de referencia para otros frames,
- un GOP (**Group Of Pictures**) siempre empieza por un I-frame.

- **P-frame (Predicted):**

- se codifica usando predicción desde frames anteriores (lista L0),
- combina bloques intra e inter según convenga.

- **B-frame (Bi-predicted):**

- se codifica usando frames anteriores y/o posteriores (listas L0 y L1),
- puede usar predicción desde el pasado, desde el futuro o desde ambos (bi-pred).

Además, esta utilidad nos generará un fichero adicional con una serie de estadísticas sobre el vídeo codificado:

```

1  RESUMEN GENERAL
2  -----
3  Total de macroblocks analizados: 281,250
4  Total de frames analizados: 250
5  Tamaño archivo original: 105,841,560 bytes
6  Tamaño archivo comprimido: 385,659 bytes
7  Ratio de compresión: 274.44:1
8  Porcentaje de compresión: 99.6%
9  Bitrate promedio: 305,452 bps (305 kbps)
10 PSNR promedio: 62.26 dB
11
12 DISTRIBUCIÓN POR TIPO DE MACROBLOCK
13 -----
14 Tipo          Cantidad  Porcentaje
15 -----
16 INTRA          33,876    12.04%
17 SKIP           21,110     7.51%
18 INTER          226,264    80.45%
19 -----
20
21 DISTRIBUCIÓN POR TIPO DE FRAME
22 -----
23 Tipo          Cantidad  Porcentaje
24 -----
25 I              3         1.20%
26 P             76        30.40%
27 B            171        68.40%
28 -----

```

Por último, la aplicación nos generará un gráfico con la evolución del bitrate y los tipos de frames, y el parámetro de cuantización (QP) a lo largo de la codificación.

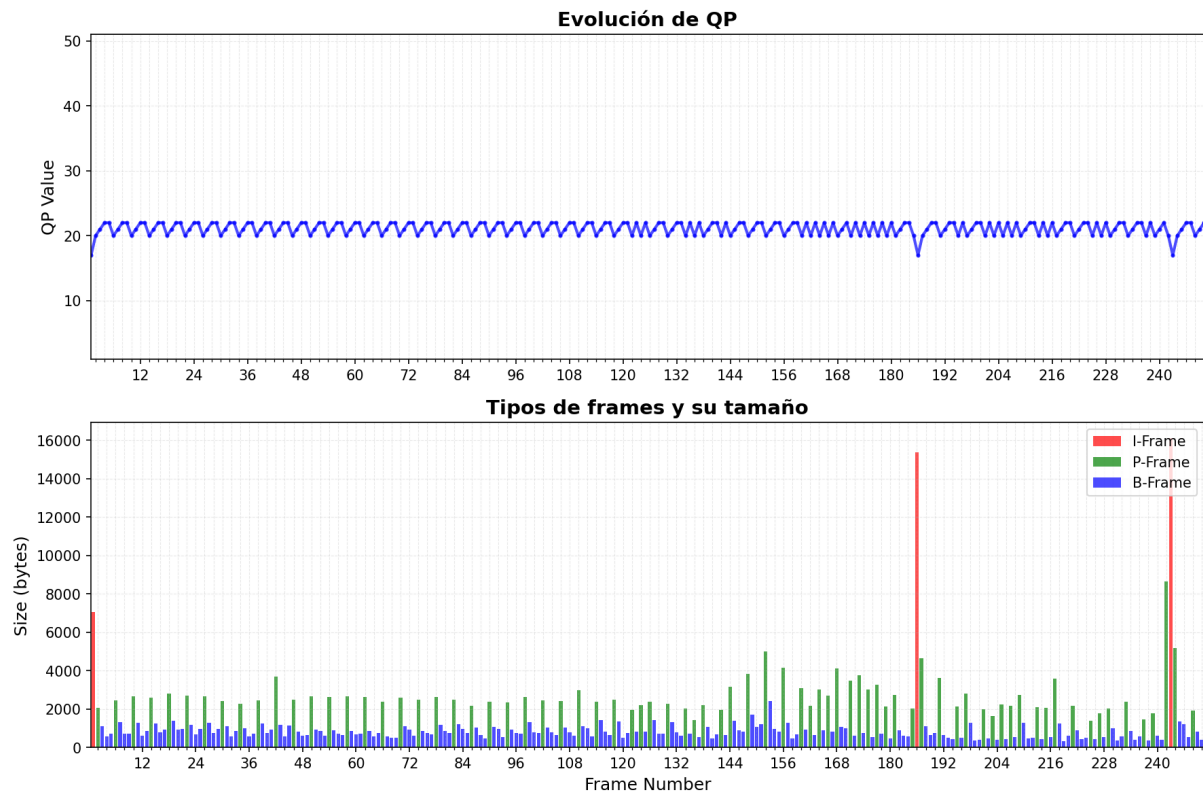


Figura 2: Gráfico de evolución del bitrate y QP

3.1.1 Ejercicio 1

Prueba a codificar tu vídeo asignado a H.264 usando la siguiente configuración de ejemplo, mediante el script `mb_visualizer.py`. Después abre el vídeo con overlay con `vlc` y comprueba que puedes ver los distintos tipos. Abre también el fichero de estadísticas para observar los datos que aparecen, y pégalo en el documento de respuestas.

```
1 python mb_visualizer.py input.y4m --output-video ej1.mp4 --params "-c:v
  libx264"
```

Donde:

- `input.y4m`: vídeo de entrada.
- `--output-video prueba_codificacion.mp4`: nombre base del vídeo codificado.
- `-c:v libx264`: codifica en H264 usando la librería x264.

3.2 Longitud del GOP

El GOP (Group Of Pictures) define una secuencia de frames en la que el primero siempre es un I-Frame.

En H264, la longitud del GOP y la distancia entre I-Frames se definen con los parámetros `keyint` y `min-keyint`.

- `keyint`: distancia máxima entre dos I-Frames.
- `min-keyint`: distancia mínima entre dos I-Frames.

En `ffmpeg` se pueden definir mediante los parámetros `-x264-params keyint=60:min-keyint=60`, donde: * `-x264-params`: son los parámetro de bajo nivel del codificador x264. * `keyint=60`: como máximo hay 60 frames entre dos I. * `min-keyint=60`: como mínimo hay 60 frames entre dos I.

3.2.1 Ejercicio 2

Vamos a probar a definir la longitud del GOP con una nueva codificación. Para ello usaremos el siguiente comando:

```
1 python mb_visualizer.py input.y4m \  
2     --output-video ej2.mp4 \  
3     --params "-c:v libx264 -x264-params keyint=10:min-keyint=10"
```

Al acabar, abre el diagrama con el tamaño y tipo de frames. Guardalo en el documento de respuestas.

Cuestión 2.1. ¿Cuántos frames ves entre cada dos I-Frames?

Cuestión 2.2. ¿Por qué aparecen los I-Frames con esa periodicidad?

Cuestión 2.3. ¿Cuáles son los tipos de frames que más ocupan en general? ¿Y los que menos? ¿Por qué?

3.2.2 Ejercicio 3

Prueba ahora a codificar una nueva version del vídeo, pero esta vez no especifiques la longitud del GOP. Obten de nuevo el diagrama con el tamaño y tipo de cada frame y guardarlo también en el documento de respuestas.

Cuestión 3.1. ¿Cuántos frames ves ahora entre cada dos I-Frames? ¿Es un número fijo o variable? ¿Podrías ver algún patrón de periodicidad en la secuencia de frames P y B?

3.3 Bitrate fijo vs Bitrate variable

A la hora de codificar un vídeo, una de las decisiones más importantes es la elección del bitrate. El bitrate es la cantidad de bits que se utilizan para codificar el vídeo por unidad de tiempo.

- **CBR (bitrate constante):**

- se fija un bitrate objetivo.
- el codificador ajusta dinámicamente el parámetro de cuantización (QP) para respetar ese bitrate.
- escenas complejas pueden verse más degradadas porque no se permite subir mucho la tasa instantánea.

- **VBR (bitrate variable):**

- se fija una calidad objetivo.
- el QP se elige para mantener más o menos constante la calidad de salida.
- el bitrate resultante se adapta al contenido (mayor en escenas complicadas, menor en escenas sencillas).

Para codificar con un bitrate constante con `ffmpeg` usaremos los parámetros:

- `-b:v 2000k` **Bitrate objetivo de vídeo:** 2000 kbps. En CBR suele coincidir con `minrate` y `maxrate` para que no varíe.
- `-minrate 2000k` **Bitrate mínimo** que el encoder intentará mantener. Si lo igualas a `-b:v`, fuerzas a que no baje de ese valor.
- `-maxrate 2000k` **Bitrate máximo** que el encoder permitirá. Igualándolo a `-b:v` limitas los picos por arriba.
- `-bufsize 4000k` **Tamaño del buffer VBV** (Video Buffering Verifier). Controla cuánta variación instantánea de bitrate se permite alrededor de `-b:v`.
 - Valores pequeños → bitrate más rígido, menos margen.
 - Valores grandes → más variación permitida.

El comando completo para codificar con un bitrate fijo será:

```
1 python mb_visualizer.py input.y4m --output-video salida.mp4 --params "-c:v libx264 -b:v 2000k -minrate 2000k -maxrate 2000k -bufsize 4000k"
```

Para codificar con un bitrate variable con `ffmpeg` usaremos el parámetro `-crf`:

- `-crf 23` **CRF = Constant Rate Factor**, controla la calidad objetivo.

- Rango típico: 18–28
- **Más bajo = mejor calidad = más bitrate**
- **Más alto = peor calidad = menos bitrate**

El comando completo para codificar con un bitrate variable será:

```
1 python mb_visualizer.py input.y4m --output-video salida.mp4 --params "--c:v libx264 -crf 23"
```

3.3.1 Ejercicio 4

Ejecuta una codificación para un bitrate fijo de 500kbps. Copia el valor de bitrate medio que aparece en las estadísticas y el tamaño final del vídeo codificado en el documento de respuestas.

Cuestión 4.1. ¿Cuál es el valor de bitrate medio que aparece en las estadísticas? ¿Es consistente con el bitrate fijo que has especificado?

Cuestión 4.2. Sabiendo que el vídeo tiene una duración de 10 segundos, ¿es consistente el tamaño final del vídeo codificado con el bitrate promedio? ¿Por qué?

3.4 Ejercicio 5

Códifica ahora el vídeo usando las siguientes configuraciones:

- CBR con bitrate fijo de 2000 kbps.
- CBR con bitrate fijo de 250 kbps.
- VBR con CRF de 40.
- VBR con CRF de 23.
- VBR con CRF de 5.

Para cada uno de ellos genera el fichero de estadísticas, y crea una tabla con los siguientes datos:

Modo	Bitrate medio	pSNR media	Tasa de compresión
CBR 2000kbps	- kbps	- dB	~:1
CBR 250kbps	- kbps	- dB	~:1
VBR CRF 40	- kbps	- dB	~:1
VBR CRF 23	- kbps	- dB	~:1
VBR CRF 5	- kbps	- dB	~:1

Cuestión 5.1. ¿Qué modo ofrece mejor calidad? ¿Y la mejor tasa de compresión? ¿Qué relación ves entre ambas?

Cuestión 5.2. Observa los valores de QP en las diferentes gráficas de cada caso. ¿Son consistentes con el tipo de bitrate (constante o variable) de cada caso? Explica tu respuesta.

Cuestión 5.4. Compara los valores de QP de los dos casos con CBR. ¿Qué observas? ¿Es consistente con el bitrate de cada caso? Explica tu respuesta.

3.5 CRF no es QP

Aunque hemos usado el parámetro `-crf` para codificar con VBR, ya que es el parámetro habitual en estos casos, en realidad su valor no es exactamente el mismo que el parámetro de cuantización (QP). CRF es una medida de la calidad del video, de forma que el codificador ajusta el QP para mantener siempre esa calidad objetivo. Por ejemplo, si un frame se codifica con pocos bits, puede reducirse el QP en ese momento. El parámetro QP es directamente el factor de cuantización, y también se puede setear en `ffmpeg` con el parámetro `-qp`.

3.6 Ejercicio 6

Realiza dos codificaciones con VBR, una con un CRF de 25 y otra con el valor `qp` también a 25. Saca las estadísticas y las gráficas de cada caso, y pon en el documento de respuestas el bitrate medio y la pSNR media de cada caso.

Cuestión 6.1. ¿Qué modo ofrece mejor calidad? ¿Y la mejor tasa de compresión? ¿Por qué crees que se obtienen estos resultados?

3.7 Usando solo bloques INTRA vs INTRA + INTER

Los bloques INTRA son en general más costosos de codificar que los INTER, que al apoyarse en la redundancia temporal pueden codificar bloques enteros con muy pocos bits. Para comprobar esto, vamos a codificar el vídeo usando solo bloques INTRA y otra vez con bloques INTRA + INTER.

Aunque `ffmpeg` no nos deja elegir una codificación solo con INTRA, podemos forzarla seteando un GOP de longitud máxima y mínima de 1 frame. De esta manera, todos los frames serán de tipo I.

3.7.1 Ejercicio 7

Revisa como se definía la longitud del GOP en `ffmpeg`, y realiza dos codificaciones con los siguientes parámetros:

- Bitrate variable con CRF de 23.
- Codificación con MB solo Intra, y otra con MB Intra + Inter.

Guarda los datos de las estadísticas en una tabla similar a la del anterior ejercicio:

Modo	Bitrate	pSNR media	Tasa de compresión
INTRA solo	- kbps	- dB	-.1
INTRA + INTER	- kbps	- dB	-.1

Haz también otra tabla con el número de bloques de cada tipo que aparecen en las estadísticas, de la forma:

Modo	% MB INTRA	% MB INTER	% MB SKIP
INTRA solo	-	-	-
INTRA + INTER	-	-	-

Observa el video con el overlay de macrobloques y comprueba que efectivamente se están usando solo bloques INTRA o INTRA + INTER en cada caso.

Cuestión 7.1. ¿Qué modo ofrece mejor calidad? ¿Y la mejor tasa de compresión? ¿Cuál es la razón por la que se obtienen los resultados en las anteriores cuestiones? Recuerda lo que mide la pSNR, y las limitaciones de la transformada usada en H.264 durante el proceso de codificación.

3.8 Vectores de movimiento

Los **vectores de movimiento** son la herramienta que usa la predicción INTER para localizar en otro frame la región que mejor coincide con el bloque actual.

Con `ffmpeg` podemos ver los vectores de movimiento con el siguiente comando:

```
1 ffmpeg -flags2 +export_mvs -i input.mp4 -vf codecview=mv=pf+bf+bb
   output.mp4
```

La opción `mv` permite ver los vectores de movimiento que serán: - `pf` – vectores de movimiento predichos hacia adelante de fotogramas P - `bf` – vectores de movimiento predichos hacia adelante de fotogramas B - `bb` – vectores de movimiento predichos hacia atrás de fotogramas B

Nota: Este comando debe usarse con la salida después de haber codificado el video y4m.

3.8.1 Ejercicio 8

Utiliza el anterior comando para codificar el video con VBR con CRF de 23 y mostrar los vectores de movimiento solo de tipo **pf** y **bf**. Haz una captura de pantalla de la imagen con los vectores de movimiento y pégala en el documento de respuesta.

Cuestión 8.1. Usa la tecla **E** de **vlc** para ver los vectores de movimiento frame a frame, y busca alguna parte de tu video donde haya algo de movimiento. ¿Hacia donde se produce el movimiento y hacia donde apuntan los vectores de movimiento según se avanzan los frames?

3.9 Métodos de búsqueda y rangos de búsqueda

Para encontrar los vectores de movimiento, el codificador busca la mejor coincidencia dentro de una **ventana de búsqueda** alrededor de la posición del bloque.

El tamaño de esta ventana se controla con **merange**. Es el rango en píxeles que se va a utilizar para buscar la mejor coincidencia.

- **merange**=4 → poco margen, búsqueda rápida pero imprecisa.
- **merange**=16 → valor típico razonable.
- **merange**=48 o más → búsqueda amplia, pero más costosa en tiempo de cómputo.

El **método de búsqueda** se define con **me**:

- **me=dia**: Usa un patrón en forma de rombo alrededor de la posición actual. Se puede quedar atrapado en mínimos locales y no encontrar el óptimo.
- **me=hex**: Usa un patrón en forma de hexágono alrededor de la posición actual. Es un buen compromiso entre velocidad y precisión. Suele ser el método por defecto.
- **me=umh**: Usa varios patrones de búsqueda en forma de hexágono alrededor de la posición actual. Es más exhaustivo que el método **me=hex**, pero es más lento.
- **me=esa**: Es el método más exhaustivo, pero es muy lento.

Por ejemplo, para codificar con un rango de búsqueda de 16 pixels y un método de búsqueda de **me=hex** se usaría el siguiente parámetro:

```
1 -x264-params "me=hex:merange=16"
```

3.9.1 Ejercicio 9

Codifica un mismo vídeo con un bitrate variable con **-qp 25**, método de búsqueda **me=hex** y variando el rango de búsqueda a 4, 16 y 48 pixels.

Para cada configuración genera una tabla con los siguientes datos:

Rango de búsqueda	Bitrate medio	pSNR media	% MB INTRA	% MB INTER	% MB SKIP
4	- kbps	- dB	-%	-%	-%
16	- kbps	- dB	-%	-%	-%
48	- kbps	- dB	-%	-%	-%

Cuestión 9.1 ¿Qué método de búsqueda ofrece mejor calidad? ¿Por qué crees que se obtienen estos resultados?

Cuestión 9.2 A la vista de los resultados, ¿valdrá la pena aumentar el rango de búsqueda al máximo para mejorar la calidad? ¿Dependerá del contenido del vídeo (tipo de escenas)? Explica tu respuesta.

3.9.2 Ejercicio 10

Ahora repite lo mismo pero cambia el método de búsqueda a **me=esa** y rellena la misma tabla de antes.

Rango de búsqueda	Bitrate medio	pSNR media	% MB INTRA	% MB INTER	% MB SKIP
4	- kbps	- dB	-%	-%	-%
16	- kbps	- dB	-%	-%	-%
48	- kbps	- dB	-%	-%	-%

Cuestión 10.1 ¿Qué diferencias observas entre los resultados de ahora y los de antes? Explica cuál crees que es la razón de estas diferencias, si las hay.